

Q46. What are Wrapper Classes? Why do we need them?

Simple Answer

Wrapper Classes are object representations of primitive data types.

Java provides a wrapper class for every primitive type, allowing primitives to be used where objects are required, such as in Collections and Generics.

Detailed Explanation

Primitive data types:

```
int  
double  
char  
boolean
```

Corresponding Wrapper Classes:

Primitive	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

Since Collections store only objects, Wrapper Classes are required.

Real-Time Example

Suppose you want to store employee IDs in an `ArrayList`.

The following is **not allowed**:

```
ArrayList<int> ids;
```

Instead, use:

```
ArrayList<Integer> ids;
```

Code Example

```
ArrayList<Integer> numbers = new ArrayList<>();
```

```
numbers.add(10);
```

```
numbers.add(20);
```

```
System.out.println(numbers);
```

Output

```
[10, 20]
```

Interview Tip

Frequently asked question:

Can an ArrayList store primitive data types?

Answer:

No.

It stores only objects, so Wrapper Classes are used.

Key Points

- ✓ Wrapper Classes convert primitives into objects.
- ✓ Required for Collections.
- ✓ Required for Generics.
- ✓ Part of `java.lang`.

Q47. What is Autoboxing and Unboxing?

Simple Answer

Autoboxing is the automatic conversion of a primitive type into its corresponding Wrapper Class.

Unboxing is the automatic conversion of a Wrapper Class object back into its primitive type.

Detailed Explanation

Before Java 5, developers had to manually convert primitives into Wrapper Classes.

Since Java 5, Java performs these conversions automatically.

Real-Time Example

When adding an `int` value into an `ArrayList<Integer>`, Java automatically converts it into an `Integer` object.

When retrieving the value, Java automatically converts it back to `int`.

Code Example

```
ArrayList<Integer> numbers = new ArrayList<>();  
  
numbers.add(100);           // Autoboxing  
  
int value = numbers.get(0);  // Unboxing  
  
System.out.println(value);
```

Output

100

Interview Tip

Remember:

Primitive → Object → **Autoboxing**

Object → Primitive → **Unboxing**

Key Points

- ✓ Introduced in Java 5.
- ✓ Makes code simpler.
- ✓ Frequently used in Collections.
- ✓ Eliminates manual conversion.

Q48. What is Serialization and Deserialization?

Simple Answer

Serialization is the process of converting an object into a byte stream so that it can be stored in a file or transmitted over a network.

Deserialization is the process of converting the byte stream back into an object.

Detailed Explanation

To serialize an object:

- The class must implement the `Serializable` interface.
- Java converts the object's state into bytes.

During deserialization, the byte stream is reconstructed into the original object.

Common use cases:

- Saving objects to files
- Sending objects across a network
- Caching
- Distributed applications

Real-Time Example

Suppose a banking application stores customer session details.

The session object can be serialized and saved.

Later, it can be deserialized to restore the user's session.

Code Example

```
import java.io.Serializable;

class Employee implements Serializable {
```

```
int id = 101;  
  
String name = "Murali";  
}
```

Interview Tip

A common interview question:

Which interface is used for Serialization?

Answer:

`Serializable`

It is a **Marker Interface**.

Key Points

- ✓ Converts objects into byte streams.
- ✓ Used for file storage and network communication.
- ✓ Requires the `Serializable` interface.
- ✓ Supports object persistence.

Q49. What is the **transient** keyword?

Simple Answer

The `transient` keyword is used to prevent a field from being serialized.

When an object is serialized, transient fields are skipped and are not saved.

Detailed Explanation

Sensitive information such as:

- Passwords
- OTPs
- Security Tokens

should not be stored in serialized files.

Declaring these fields as `transient` ensures they are excluded during serialization.

Real-Time Example

Suppose an `Employee` object contains:

- Employee ID
- Name
- Password

The password should never be written to a file during serialization.

Code Example

```
import java.io.Serializable;

class Employee implements Serializable {

    int id = 101;

    transient String password = "Admin123";
}
```

Interview Tip

Interviewers often ask:

Can a transient variable be serialized?

Answer:

No.

It is ignored during serialization.

Key Points

- ✓ Used with Serialization.
- ✓ Prevents sensitive data from being stored.
- ✓ Commonly used for passwords and tokens.
- ✓ Improves security.

Q50. What is a Marker Interface? Give examples.

Simple Answer

A **Marker Interface** is an interface that does not contain any methods or fields.

It is used to provide metadata or special instructions to the JVM or framework.

Detailed Explanation

Marker Interfaces are empty interfaces.

They indicate that a class has a special capability.

Common examples:

- `Serializable`
- `Cloneable`
- `Remote`

These interfaces are recognized by the JVM or frameworks to enable specific behavior.

Real-Time Example

When an object implements `Serializable`, Java knows that the object is allowed to be serialized.

If the interface is not implemented, attempting serialization results in a `NotSerializableException`.

Code Example

```
import java.io.Serializable;

class Employee implements Serializable {

}
```

Interview Tip

A common interview question:

Why is `Serializable` called a Marker Interface?

Answer:

Because it has **no methods** but marks a class as eligible for serialization.

Key Points

- ✓ Marker Interfaces contain no methods.
- ✓ Provide metadata.
- ✓ Used by the JVM and frameworks.
- ✓ `Serializable` and `Cloneable` are common examples.

opic 4: Exception Handling

Set 1 (Questions 51–55)

Q51. What is an Exception in Java?

Simple Answer

An **Exception** is an event that occurs during program execution and disrupts the normal flow of the program.

Java provides an exception handling mechanism to handle such runtime errors gracefully without abruptly terminating the application.

Detailed Explanation

Exceptions occur due to situations such as:

- Division by zero
- Null object access
- Invalid array index
- File not found
- Database connection failure

If exceptions are not handled, the program terminates unexpectedly.

Java uses the following keywords for exception handling:

- try
- catch
- finally
- throw
- throws

Real-Time Example

Consider an online banking application.

If a user enters an invalid account number, the application should display an error message instead of crashing.

Exception handling allows the application to continue running smoothly.

Code Example

```
public class Main {  
  
    public static void main(String[] args) {  
  
        try {  
  
            int result = 10 / 0;  
  
        } catch (ArithmeticException e) {  
  
            System.out.println("Cannot divide by  
zero.");  
        }  
  
        System.out.println("Program  
continues...");  
    }  
}
```

```
}
```

Output

```
Cannot divide by zero.  
Program continues...
```

Interview Tip

Remember:

Exception → Can be handled.

Error → Generally cannot be handled (e.g., `OutOfMemoryError`, `StackOverflowError`).

Interviewers often ask this as a follow-up.

Key Points

- ✓ Disrupts normal program execution.
- ✓ Can be handled using `try-catch`.
- ✓ Prevents abnormal program termination.
- ✓ Improves application reliability.

Q52. What is the difference between Checked and Unchecked Exceptions?

Simple Answer

Checked Exceptions are checked by the compiler and must be handled or declared.

Unchecked Exceptions occur at runtime and are not checked by the compiler.

Detailed Explanation

Checked Exception

Handled at compile time.

Examples:

- `IOException`
- `SQLException`
- `FileNotFoundException`

The compiler forces the programmer to handle these exceptions.

Unchecked Exception

Occurs during runtime.

Examples:

- `NullPointerException`
- `ArithmeticException`
- `ArrayIndexOutOfBoundsException`

These are usually caused by programming mistakes.

Comparison

Checked Exception	Unchecked Exception
Compile-time	Runtime
Must be handled	Handling is optional
Compiler checks	Compiler does not check
Extends	Extends

Exception	RuntimeException
-----------	------------------

Real-Time Example

Reading a file:

- File may not exist → Checked Exception.

Accessing a null object:

- Programming error → Unchecked Exception.

Code Example

```
import java.io.*;

public class Main {

    public static void main(String[] args) {

        try {

            FileReader file = new
FileReader("data.txt");

            } catch (IOException e) {

                System.out.println("File not
found.");
            }

        }
    }
}
```

Interview Tip

Remember:

Checked = Compiler checks

Unchecked = Programmer checks

Key Points

- ✓ Checked Exceptions occur at compile time.
- ✓ Unchecked Exceptions occur at runtime.
- ✓ Checked Exceptions must be handled or declared.
- ✓ RuntimeException and its subclasses are unchecked.

Q53. What is the difference between **throw** and **throws**?

Simple Answer

Both `throw` and `throws` are used in exception handling, but they serve different purposes.

- `throw` is used to explicitly throw an exception.
- `throws` is used to declare that a method may throw one or more exceptions.

Detailed Explanation

throw

- Used inside a method.
- Throws one exception at a time.
- Followed by an exception object.

Example:

```
throw new IllegalArgumentException("Invalid  
Age");
```

throws

- Used in the method signature.
- Declares possible exceptions.
- Can declare multiple exceptions.

Example:

```
public void readFile() throws IOException
```

Comparison

throw	throws
Used inside a method	Used in the method declaration
Throws an exception	Declares exceptions
One exception	One or more exceptions

Real-Time Example

A voting application checks a person's age.

If the age is below 18, the application throws an exception.

The calling method is informed using `throws`.

Code Example


```
public class Main {  
  
    static void checkAge(int age) {  
  
        if (age < 18) {  
  
            throw new  
IllegalArgumentException("Not eligible to  
vote.");  
        }  
  
        System.out.println("Eligible");  
    }  
  
    public static void main(String[] args) {  
  
        checkAge(16);  
    }  
}
```

Interview Tip

A common interview question:

Can we use `throw` without `throws`?

Yes, especially for unchecked exceptions.

Key Points

✓ `throw` throws an exception.

✓ `throws` declares exceptions.

✓ `throw` is followed by an object.

✓ `throws` is used in the method signature.

Q54. What is the purpose of the **finally** block?

Simple Answer

The `finally` block contains code that executes after the `try` and `catch` blocks, regardless of whether an exception occurs.

It is commonly used to release resources such as files, database connections, or network sockets.

Detailed Explanation

The `finally` block executes:

- When no exception occurs.
- When an exception occurs and is caught.
- Even when a `return` statement is executed inside the `try` block.

It may not execute only in rare cases, such as when the JVM terminates unexpectedly or `System.exit()` is called before reaching it.

Real-Time Example

Suppose a banking application opens a database connection.

Whether the transaction succeeds or fails, the connection should always be closed.

The cleanup code belongs in the `finally` block.

Code Example

```
try {  
  
    System.out.println("Inside try");  
  
} finally {  
  
    System.out.println("Closing resources");  
}
```

Output

```
Inside try  
Closing resources
```

Interview Tip

In modern Java, **try-with-resources** is often preferred over `finally` for resources that implement `AutoCloseable`.

Key Points

- ✓ Executes regardless of exceptions.
- ✓ Used for resource cleanup.
- ✓ Almost always executes.
- ✓ Complements `try-catch`.

Q55. What is a Custom Exception? Why do we create one?

Simple Answer

A **Custom Exception** is a user-defined exception created by extending the `Exception` or `RuntimeException` class.

It is used to represent business-specific errors that are not covered by Java's built-in exceptions.

Detailed Explanation

Custom Exceptions improve code readability and make error handling more meaningful.

Examples:

- `InvalidEmployeeException`
- `InsufficientBalanceException`
- `AgeNotEligibleException`

By creating a custom exception, the application can clearly communicate business rule violations.

Real-Time Example

In an ATM application, if a customer tries to withdraw more money than the available balance, the application throws an `InsufficientBalanceException`.

This exception clearly describes the business problem.

Code Example

```
class InsufficientBalanceException extends
Exception {

    public InsufficientBalanceException(String
message) {

        super(message);
    }
}

public class Main {

    static void withdraw(double balance, double
amount)
        throws InsufficientBalanceException {

        if (amount > balance) {

            throw new
InsufficientBalanceException("Insufficient
Balance");
        }

        System.out.println("Withdrawal
Successful");
    }

    public static void main(String[] args) {

        try {

            withdraw(5000, 7000);

        } catch (InsufficientBalanceException e)
{
```

```
        System.out.println(e.getMessage());
    }
}
```

Output

Insufficient Balance

Interview Tip

A common interview question:

Should a Custom Exception extend `Exception` or `RuntimeException`?

- Extend **`Exception`** for checked exceptions.
- Extend **`RuntimeException`** for unchecked exceptions.

Choose based on whether callers should be forced to handle the exception.

Key Points

- ✓ Created for business-specific errors.
- ✓ Improves code readability.
- ✓ Can extend `Exception` or `RuntimeException`.
- ✓ Widely used in enterprise applications.

Q56. What is Try-with-Resources? Why is it used?

Simple Answer

Try-with-Resources is a feature introduced in Java 7 that automatically closes resources such as files, database connections, and streams after they are used.

It eliminates the need to manually close resources in the `finally` block.

Detailed Explanation

Before Java 7:

Developers had to close resources manually.

```
finally{  
    file.close();  
}
```

If developers forgot to close the resource, it could lead to:

- Memory leaks
- Resource leaks
- File locking issues

With Try-with-Resources:

Resources that implement the **AutoCloseable** interface are automatically closed.

Real-Time Example

Suppose an application reads an employee data file.

After reading the file, the file should always be closed.

Instead of writing cleanup code inside `finally`, Java automatically closes the file.

Code Example

```
import java.io.*;

public class Main {

    public static void main(String[] args) {

        try (BufferedReader br =
            new BufferedReader(new
FileReader("data.txt"))) {

            System.out.println(br.readLine());

        } catch (IOException e) {

            System.out.println(e.getMessage());

        }

    }

}
```

Interview Tip

Interview Question:

Why is Try-with-Resources preferred over finally?

Answer:

Because Java automatically closes resources, making the code cleaner and less error-prone.

Key Points

✓ Introduced in Java 7.

- ✓ Automatically closes resources.
- ✓ Eliminates resource leaks.
- ✓ Works with AutoCloseable.

Q57. What is the Exception Hierarchy in Java?

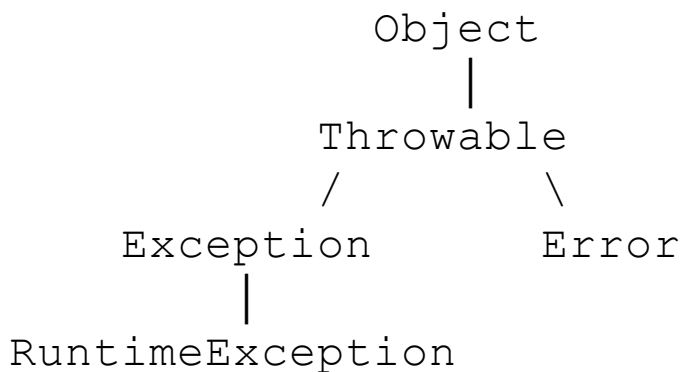
Simple Answer

All exceptions in Java inherit from the **Throwable** class.

Throwable has two main subclasses:

- Exception
- Error

Detailed Explanation



Exception

Represents conditions that applications can handle.

Examples:

- IOException

- SQLException

RuntimeException

Occurs due to programming mistakes.

Examples:

- NullPointerException
- ArithmeticException

Error

Represents serious system-level problems.

Examples:

- OutOfMemoryError
- StackOverflowError

Applications generally should not attempt to recover from these.

Real-Time Example

Suppose a banking application attempts to open a file.

If the file is missing:

`IOException`

If memory is exhausted:

`OutOfMemoryError`

Code Example

```
try {  
  
    String name = null;  
  
    System.out.println(name.length());  
  
} catch (NullPointerException e) {  
  
    System.out.println("Null Object");  
}
```

Interview Tip

One of the favorite interview questions:

Does RuntimeException extend Exception?

Yes.

RuntimeException

↓

Exception

↓

Throwable

Key Points

- ✓ Throwable is the root class.
- ✓ Exception represents recoverable problems.
- ✓ Error represents serious JVM issues.
- ✓ RuntimeException is an unchecked exception.

Q58. What are the Best Practices for Exception Handling?

Simple Answer

Good exception handling improves application reliability and makes debugging easier.

Developers should handle exceptions properly instead of hiding or ignoring them.

Detailed Explanation

Best Practices

✓ Catch Specific Exceptions

Instead of:

```
catch (Exception e)
```

Prefer:

```
catch (IOException e)
```

✓ Don't Ignore Exceptions

Never write:

```
catch (Exception e) {  
  
}
```

Always log or handle the exception.

✓ Use Try-with-Resources

Automatically closes resources.

✓ Create Custom Exceptions

Use meaningful business exceptions.

Example:

```
InvalidEmployeeException
```

✓ Don't Use Exceptions for Normal Program Flow

Avoid:

```
try{  
  
    // normal logic  
  
}catch(Exception e){  
  
}
```

Exceptions should represent exceptional situations.

Real-Time Example

A payment application should not throw a generic `Exception`.

Instead:

`InvalidPaymentException`

`PaymentTimeoutException`

`InsufficientBalanceException`

These provide meaningful error information.

Code Example

```
try{  
  
    processPayment();  
  
}catch(InsufficientBalanceException e){  
  
    System.out.println(e.getMessage());  
  
}
```

Interview Tip

One of the best answers you can give:

Exceptions are for **exceptional situations**, not for controlling normal program logic.

This demonstrates good coding practices.

Key Points

- ✓ Catch specific exceptions.
- ✓ Never ignore exceptions.
- ✓ Use Try-with-Resources.
- ✓ Create meaningful Custom Exceptions.
- ✓ Log exceptions properly.

Q46. What are Wrapper Classes? Why do we need them?

Simple Answer

Wrapper Classes are object representations of primitive data types.

Java provides a wrapper class for every primitive type, allowing primitives to be used where objects are required, such as in Collections and Generics.

Detailed Explanation

Primitive data types:

```
int
double
char
boolean
```

Corresponding Wrapper Classes:

Primitive	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

Since Collections store only objects, Wrapper Classes are required.

Real-Time Example

Suppose you want to store employee IDs in an `ArrayList`.

The following is **not allowed**: